

Agent Phantom Recovery

12-supabase-schema.md

Production Supabase Schema Architecture

Version: 1.0

Status: Production Design

1. Purpose

This document defines how Agent Phantom Recovery will be implemented using Supabase as the primary backend platform.

Unlike the generic database schema, this document focuses specifically on:

- Supabase PostgreSQL
- Supabase Auth
- Supabase Storage
- Supabase Realtime
- Supabase Edge Functions
- Supabase Row Level Security (RLS)
- Supabase Vector Support (pgvector)
- Supabase Migrations
- Antigravity IDE Integration

This document serves as the implementation blueprint for production deployment.

2. Why Supabase

Supabase provides several advantages for Agent Phantom Recovery.

Benefits

Managed PostgreSQL

Provides:

- ACID compliance
- Transactions
- Extensions

- Reliability
-

Authentication

Built-in support for:

- Email/password
 - OAuth
 - Magic links
 - Session management
-

Realtime

Required for:

- Live terminal streaming
 - Task progress
 - Browser events
 - Agent updates
-

Storage

Required for:

- Reports
 - Logs
 - Screenshots
 - Repository artifacts
-

pgvector

Allows:

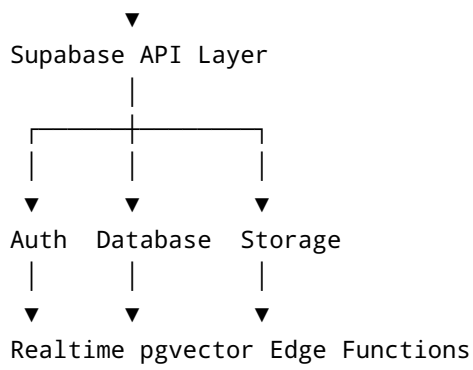
- Memory retrieval
- Semantic search
- Repository chunk search

without requiring a separate vector database during MVP.

3. Supabase Architecture

```
Antigravity IDE
```

```
|
```



4. Schema Naming Convention

All tables should follow:

snake_case

Example:

task_executions
project_memories
repository_indexes

Primary Keys

All tables:

UUID

Timestamps

Every table:

created_at
updated_at

5. Authentication Schema

Supabase Auth manages identity.

auth.users

Managed by Supabase.

Do not modify directly.

profiles

profiles

Column	Type
id	UUID
username	TEXT
avatar_url	TEXT
role	TEXT
created_at	TIMESTAMP
updated_at	TIMESTAMP

Relationship

auth.users
↓
profiles

6. Organizations (Future Ready)

organizations

Column	Type
id	UUID

Column	Type
name	TEXT
owner_id	UUID
created_at	TIMESTAMP

Purpose

Supports future:

- teams
- enterprise
- shared workspaces

7. Projects Schema

projects

Column	Type
id	UUID
owner_id	UUID
name	TEXT
description	TEXT
visibility	TEXT
created_at	TIMESTAMP
updated_at	TIMESTAMP

Indexes

idx_projects_owner

8. Repositories Schema

repositories

Column	Type
id	UUID
project_id	UUID
source_url	TEXT
default_branch	TEXT
language	TEXT
framework	TEXT
file_count	INTEGER
created_at	TIMESTAMP

Indexes

idx_repository_project

9. Tasks Schema

Core table.

tasks

Column	Type
id	UUID
project_id	UUID
goal	TEXT
status	TEXT
priority	INTEGER
current_plan_id	UUID
created_at	TIMESTAMP

Column	Type
updated_at	TIMESTAMP

Status Values

CREATED
 PLANNING
 COLLECTING_EVIDENCE
 ANALYZING
 EXECUTING
 VERIFYING
 COMPLETED
 FAILED
 BLOCKED
 PAUSED

Indexes

idx_task_status
 idx_task_project
 idx_task_priority

10. Plans Schema

plans

Column	Type
id	UUID
task_id	UUID
version	INTEGER
summary	TEXT
status	TEXT
created_at	TIMESTAMP

11. Plan Steps Schema

plan_steps

Column	Type
id	UUID
plan_id	UUID
step_order	INTEGER
description	TEXT
tool_name	TEXT
status	TEXT

Example

Run Tests
Inspect Logs
Analyze Code
Generate Patch
Verify

12. Evidence Schema

evidence

Column	Type
id	UUID
task_id	UUID
source	TEXT
evidence_type	TEXT
content	JSONB
confidence	NUMERIC
created_at	TIMESTAMP

Why JSONB

Evidence structures vary significantly.

Examples:

- logs
- browser observations
- code snippets
- search results

JSONB provides flexibility.

13. Findings Schema

findings

Column	Type
id	UUID
task_id	UUID
summary	TEXT
severity	TEXT
confidence	NUMERIC
status	TEXT
created_at	TIMESTAMP

14. Patches Schema

patches

Column	Type
id	UUID
task_id	UUID
finding_id	UUID
summary	TEXT
status	TEXT

Column	Type
created_at	TIMESTAMP

Status

DRAFT
GENERATED
TESTED
VERIFIED
REJECTED

15. Patch Files Schema

patch_files

Column	Type
id	UUID
patch_id	UUID
file_path	TEXT
diff_summary	TEXT

16. Verification Schema

verifications

Column	Type
id	UUID
task_id	UUID
patch_id	UUID
status	TEXT
confidence	NUMERIC
created_at	TIMESTAMP

17. Verification Checks Schema

verification_checks

Column	Type
id	UUID
verification_id	UUID
check_type	TEXT
result	TEXT
details	JSONB

18. Reports Schema

reports

Column	Type
id	UUID
task_id	UUID
report_type	TEXT
storage_path	TEXT
created_at	TIMESTAMP

19. Execution Schema

Tracks every action.

executions

Column	Type
id	UUID
task_id	UUID
action	TEXT

Column	Type
status	TEXT
started_at	TIMESTAMP
completed_at	TIMESTAMP

20. Tool Executions Schema

tool_executions

Column	Type
id	UUID
task_id	UUID
tool_name	TEXT
status	TEXT
output_summary	TEXT
duration_ms	INTEGER
created_at	TIMESTAMP

21. Browser Sessions Schema

Required for Antigravity IDE.

browser_sessions

Column	Type
id	UUID
task_id	UUID
current_url	TEXT
created_at	TIMESTAMP

22. Browser Observations Schema

browser_observations

Column	Type
id	UUID
browser_session_id	UUID
observation	TEXT
metadata	JSONB
created_at	TIMESTAMP

23. Terminal Sessions Schema

terminal_sessions

Column	Type
id	UUID
task_id	UUID
environment_id	UUID
created_at	TIMESTAMP

24. Terminal Commands Schema

terminal_commands

Column	Type
id	UUID
terminal_session_id	UUID
command	TEXT
output	TEXT
exit_code	INTEGER

Column	Type
created_at	TIMESTAMP

25. Event Store Schema

Event sourcing support.

events

Column	Type
id	UUID
task_id	UUID
event_type	TEXT
payload	JSONB
created_at	TIMESTAMP

Example Events

TaskCreated
TaskStarted
ToolStarted
ToolFinished
VerificationPassed

26. Checkpoints Schema

Long-horizon execution support.

checkpoints

Column	Type
id	UUID
task_id	UUID
snapshot	JSONB

Column	Type
created_at	TIMESTAMP

27. Memory Architecture

Memory is split into:

Metadata
+
Embeddings

Memory Records

memory_records

Column	Type
id	UUID
project_id	UUID
memory_type	TEXT
importance	NUMERIC
created_at	TIMESTAMP

28. pgvector Memory Collections

Supabase PostgreSQL supports:

vector

via pgvector.

Project Memories

project_memories

Column	Type
id	UUID
project_id	UUID
content	TEXT
embedding	VECTOR
importance	NUMERIC
created_at	TIMESTAMP

Experience Memories

experience_memories

Column	Type
id	UUID
strategy	TEXT
outcome	TEXT
embedding	VECTOR
usefulness_score	NUMERIC

29. Repository Embeddings

Repository intelligence support.

repository_chunks

Column	Type
id	UUID
repository_id	UUID
file_path	TEXT
chunk_content	TEXT
embedding	VECTOR
created_at	TIMESTAMP

Enables

- semantic code search
 - architecture retrieval
 - repository memory
-

30. Supabase Storage Buckets

reports

Stores:

PDF Reports
Markdown Reports
Task Summaries

screenshots

Stores:

Browser Screenshots
Visual Evidence

logs

Stores:

Execution Logs
Debug Logs

artifacts

Stores:

Repository Artifacts
Generated Outputs

31. Supabase Realtime

Realtime channels required.

Task Updates

task_updates

Used for:

- progress updates
 - task status changes
-

Terminal Stream

terminal_stream

Used for:

- live command output
-

Browser Stream

browser_stream

Used for:

- browser events
 - navigation updates
-

Agent Stream

agent_stream

Used for:

- planner updates
- reasoning summaries
- verification events

32. Edge Functions Architecture

Edge Functions should handle:

Repository Ingestion

repository_ingest

Memory Embedding Generation

memory_embed

Repository Chunk Embedding

repository_embed

Report Generation

generate_report

Task Export

export_task

33. Row Level Security (RLS)

All user-facing tables must use RLS.

Projects Policy

Users can access only their projects.

Tasks Policy

Users can access only tasks belonging to projects they own.

Memories Policy

Users can access only their memory records.

Reports Policy

Users can access only their reports.

34. Recommended Migration Order

1. profiles
 2. projects
 3. repositories
 4. tasks
 5. plans
 6. evidence
 7. findings
 8. patches
 9. verifications
 10. reports
 11. executions
 12. events
 13. memory tables
 14. pgvector tables
 15. storage buckets
 16. RLS policies
-

35. Antigravity IDE Mapping

Direct mapping between Supabase and IDE.

Browser Workspace

```
browser_sessions  
browser_observations
```

Terminal Workspace

```
terminal_sessions  
terminal_commands
```

Timeline Workspace

```
events  
executions
```

Source Code Workspace

```
repositories  
repository_chunks
```

Memory Workspace

```
project_memories  
experience_memories
```

Agent Workspace

```
tasks  
plans
```

findings
reports

36. Production Scaling Strategy

MVP:

Supabase PostgreSQL
+
pgvector
+
Storage
+
Realtime

Scale Phase:

Supabase PostgreSQL
+
Dedicated Qdrant
+
Redis
+
Kafka/Event Bus

Enterprise Phase:

Supabase
+
Dedicated Vector Cluster
+
Dedicated Queue Cluster
+
Dedicated Object Storage

37. Final Supabase Architecture Summary

Agent Phantom Recovery uses Supabase as the operational backbone.

The architecture combines:

```
Supabase Auth
+
PostgreSQL
+
pgvector
+
Storage
+
Realtime
+
Edge Functions
```

to support:

- Long-horizon agent execution
- Repository intelligence
- Memory retrieval
- Browser workspaces
- Terminal workspaces
- Verification pipelines
- Real-time Antigravity IDE synchronization

This schema is optimized for rapid MVP development while remaining extensible enough to support future production-scale deployments.